

- Load drivers
 - Drivers from firmware volumes (FVs) loaded into DRAM.
- Device discovery
 - PCI, USB, SATA, GPU, network initialised.
 - Console devices (keyboard, graphics, serial) ready.
- Handle database
 - Devices are represented by Handles.
 - Handles are linked to protocols (APIs by GUIDs).
 - Ex: `EFI_BLOCK_IO_PROTOCOL`, `EFI_GRAPHICS_OUTPUT_PROTOCOL`.
- Boot services
 - Memory allocation
 - Image loading
 - Protocol lookup
 - Event + timer services
- Console setup
 - Enables BIOS/UEFI setup UI (keyboard + graphics output).

In the handoff, after all drivers are initialised, boot manager and boot device selection (BDS) are invoked.

BDS phase (Boot device selection/Boot manager)

The boot manager (part of DXE) runs this phase. Key actions are:

- Read boot configuration from NVRAM variables:
 - `BootOrder`, `Boot####`.
- Select boot device (disk, USB, PXE, etc).
- Locate OS loader in EFI System Partition (ESP):
 - `bootmgfw.efi` (Windows)
 - `grubx64.efi` (Linux)
- Secure Boot verification (if enabled).
- Load + execute OS loader as a UEFI application.

OS loader execution

Examples are:

- Windows → `bootmgfw.efi`
 - Linux → `grubx64.efi` / `shimx64.efi`
- Key actions are:

BIOS vs UEFI

Feature	BIOS	UEFI
Initialisation	POST (monolithic)	Modular phases (SEC, PEI, DXE, BDS)
Execution mode	16-bit real mode	32-bit or 64-bit protected/long mode
Storage scheme	MBR (2 TB disk limit)	GPT (supports huge disks, >2 TB)
Bootloader	First 512 bytes (MBR)	EFI System Partition (ESP, FAT32)
Drivers	BIOS-internal only	Modular UEFI drivers loaded dynamically
Security	None	Secure Boot, TPM, Boot Guard supported
Flexibility	Limited	Extensible (protocols, drivers, apps)

- Use UEFI Boot Services to:
 - Read kernel + `initramfs` from disk.
 - Access graphics/network for menus.
- Load kernel image + `initramfs` into DRAM.
- Call `GetMemoryMap()` → snapshot of memory layout.
- Call `ExitBootServices()` → firmware boot services shut down.

OS kernel takes control

In the handoff, the loader jumps to the kernel entry point. The CPU is now fully under kernel control.

Kernel initialisation is as follows:

- Setup: CPU mode, paging, IDT, memory management.
 - Initialise drivers + devices.
 - Start scheduler, kernel threads.
 - Mount `initramfs`, then switch to real root FS.
- UEFI runtime services still available are:
- `GetTime()`, `SetTime()`
 - `GetVariable()`, `SetVariable()` (NVRAM)
 - `ResetSystem()`

Entering user space

Kernel spawns `init` process (PID 1):

- Linux → `/sbin/init` (`systemd/upstart`).
 - Windows → `smss.exe`.
- Init* responsibilities are:
- Mount filesystems.
 - Start background services.
 - Launch login services (CLI/GUI).
- User space is now fully alive. The user sees the login prompt/desktop. OS operation is normal from here.

The UEFI boot process is a modern, modular, and secure replacement for the legacy BIOS booting sequence. BIOS is simple but limited — it runs in 16-bit real mode, uses MBR, and has no built-in security. UEFI brings flexibility and power. It runs in 32/64-bit mode, uses a phased approach (SEC → PEI → DXE → BDS), and supports GPT disks with larger storage. It offers standardised services (boot services and runtime services), and strong security (Secure Boot, TPM).

Ultimately, UEFI bridges the gap between firmware and OS. It securely initialises hardware, loads drivers, hands off to the OS loader, and fades into the background while leaving behind only minimal runtime services.  **END**

 By: Janani Manikandan

The author is an open source contributor and works at a multinational corporation.